

NUTRICHECK

AI Nutrition Tracker — Voice-First Food Logging Platform

Product Requirements Document

Contents

1. [System Overview](#)
 2. [Onboarding & Targets Module](#)
 3. [Logging Module — the voice and text route](#)
 4. [Search & Corpus Module](#)
 5. [Today Module](#)
 6. [Ideas Module](#)
 7. [Insights & History Module](#)
 8. [Account & Settings Module](#)
 9. [Design System](#)
 10. [AI Layer](#)
 11. [Background Behaviour & Automations](#)
 12. [Database Schema \(PostgreSQL\)](#)
 13. [User Roles & Permissions](#)
 14. [Integration Specifications](#)
 15. [Build Order & Status](#)
 16. [Acceptance Criteria](#)
 17. [Open Items & Decisions](#)
 18. [Approvals & Sign-off](#)
-

1 System Overview

Document control

Field	Details
Project Name	NutriCheck — AI Nutrition Tracker (voice-first food logging)
Company	My Health School (<i>inferred from the repository owner — confirm before sign-off</i>)
Version	1.0 — full current-system specification
Scope	The entire product: mobile client (onboarding, logging, day view, ideas, insights, account) and the API behind it (auth, corpus, goals, logs, AI routes, transcription, quota)
Tech Stack	React Native 0.81 CLI (bare) + TypeScript · NestJS on the Fastify adapter · PostgreSQL 16 with <code>pgvector</code> + <code>pg_trgm</code> · Redis · Drizzle ORM · Zod as the single contract source · OpenAI-compatible model provider (<code>gpt-4o-mini</code> , Anthropic pluggable) · Gemini (speech-to-text) · Docker + Turborepo · AWS Lightsail + Caddy
Build Method	Claude Code (in-house). Two repositories sharing one contracts package and one database: <code>nutrheck-api/</code> (npm workspace monorepo) and <code>nutrheck/</code> (React Native app). Deploy by push to <code>staging</code>
Date	28 August 2026
Status	Pre-release. Backend live on staging at <code>https://3-6-120-121.sslip.io</code> ; Android build verified end-to-end on a physical device. Not yet submitted to either store

1.1 What is NutriCheck?

NutriCheck is a daily nutrition tracker for calories, protein, carbohydrate, fat and fibre. Its bet is that people quit calorie trackers because logging is work, and that language removes the work: you say or type one ordinary sentence — *"rendu dosai, chutney and a bowl of sambar"* — and the day updates.

It is built for users who eat South Indian food and speak Tamil, English or Tanglish, which is the constraint that shaped almost every technical decision in this document. A US-centric food database and English punctuation both fail on that input in ways that are invisible until a real user tries them.

The product exists to answer one question honestly, four times a day: **where did that number come from?** Every screen either shows a measured value, an estimate marked as an estimate, or a blank — never a zero standing in for something unknown.

1.2 Module map

Group	Module / Screen	Purpose
Onboarding	Welcome	Brand mark, one headline, one line of subcopy, both doors (sign in / sign up)
Onboarding	Auth — email → password	One flow for both doors; <code>check-email</code> decides which screen step two is

Group	Module / Screen	Purpose
Onboarding	Name · Profile · Activity · Objective · Rate	Five steps, one question each; nothing asked that feeds no calculation except the name
Onboarding	Targets	The payoff screen — five numbers, editable, with the reasoning visible
Logging	Ask sheet	The microphone, raised over the live tab from the centre button
Logging	Listen	Onboarding's full-screen microphone — one orb, nothing else
Logging	Type	The keyboard half of the same route, with remembered sentences
Logging	Meal details	What the model made of the words, read back before anything is logged
Logging	Composer · Confirm sheet	The corpus resolver route: draft, skeleton rows, per-item confirm
Logging	Search · Portion · Create food	The route with no model in it — the floor under everything else
Day	Today	Calorie ring counting down, five meters, meal cards, recents, undo
Day	Entry detail	One logged meal — items, portions, edit, delete
Day	Calendar	Every day of the month, logged or not, with adherence
Guidance	Ideas	Foods that fit what is left of the day
Guidance	Insights	Week view — averages over logged days, streak, per-nutrient trend
Account	You · Profile editor · Goal editor	Profile, targets, password, legal, sign out
Service	Resolver · AI-meal · Transcription · Ideas · Insight · Targets suggest	The six model-backed capabilities, each with its own guardrails (\$10)
Service	Corpus & ingest	13,440 foods, 36,768 portions, 540 aliases; USDA + curated Indian dishes
Service	Quota & spend ceiling	Per-user daily resolve quota and a per-user daily dollar cap

1.3 Phase structure

The plan is five milestones over eleven weeks. **M0, M1 and M2 are complete**; M3 is partly built (week summary and calendar exist, weight tracking does not); M4 — hardening, offline persistence, store submission — is not started. Section 15 records this stage by stage.

1.4 End-to-end flow

1. **Install and sign in.** Email and password only. `POST /v1/auth/check-email` answers whether the address is known, so step two is a real sign-in screen or a real sign-up screen rather than one hedging between them.
2. **Answer five questions.** Name; sex, age, height, weight; activity level (six options in plain language, not multipliers); objective (lose / stay / gain); and, where the objective implies one, a rate.
3. **See the targets.** Mifflin–St Jeor computes a complete answer first. The model is then shown that answer and asked whether it should move; `clampTargets` bounds whatever comes back, and every correction the server makes is returned and displayed.
4. **Log the first meal by speaking.** Onboarding ends in the microphone, not in search — the first thing after five screens of questions is the app asking what you ate.
5. **Speak or type one sentence.** The device records 16 kHz mono AAC through its own native recorder; `POST /v1/transcribe` returns **text, never a draft**, so the user reads the words before anything is interpreted.
6. **Interpret.** `POST /v1/ai-meal` reads the whole sentence in one model call and returns foods with per-100 g rates and a gram weight. It searches no corpus.
7. **Read it back.** Meal details shows the items, the portions and the totals with an estimate banner above them. Nothing is committed until the user says so.
8. **Commit.** `POST /v1/logs` is idempotent on a client-generated `clientId`; the server does the arithmetic and **freezes every nutrient on the row**. A later corpus re-ingest cannot rewrite a Tuesday in March.
9. **The day updates.** The ring counts down, the meal card carries a one-line insight, and Ideas and Insights re-read the same day view the ring was drawn from.

2 Onboarding & Targets Module

Ninety seconds, ending on a number the user wanted rather than a permission dialog they did not.

2.1 Steps

Step	Asks	Rules
Welcome	Nothing	Four elements: mark, headline, one line of subcopy, action block. Names both doors
Auth — email	Email address	<code>POST /v1/auth/check-email</code> . One screen serves sign-in and sign-up
Auth — password	Password	Minimum 6 characters, maximum 200. Length only — no composition rules. Registers or signs in depending on step one
Name	First name (required), surname (optional)	The one question that feeds no calculation. Asked on its own screen so the first thing asked of a person is not their body
Profile	Sex, age, height, weight	Metric only. There is no unit toggle; the stored value was always metric

Step	Asks	Rules
Activity	One of six levels	Plain language — "desk job, little exercise", not "1.2×". Factors 1.2 → 2.0 (athlete)
Objective	Lose · Stay · Gain	
Rate	How fast	Only shown where there is a rate to pick; maintaining goes straight to targets
Targets	Five numbers	Calories, protein, carbohydrate, fat, fibre — every one editable, with the basis shown

No step counter and no subtitles: each screen asks one thing, and a screen that asks one thing does not need to explain itself.

2.2 Target calculation

Target	Basis
BMR	Mifflin–St Jeor
TDEE	BMR × activity factor (1.2 sedentary → 2.0 athlete)
Calories	TDEE ± the chosen rate, floored at BMR , and the UI says so rather than silently clipping
Protein	Bounded per kilogram of bodyweight, not absolutely
Fat	25% of calories — policy, not derivation , so the share used is stored on <code>goals.basis.fatPctOfKcal</code> and an old target can still explain itself after the default moves
Carbohydrate	The remainder of the calorie budget
Fibre	14 g per 1000 kcal

Goals are **append-only** with `effective_from`. A day view resolves the goal that was in effect on that date, never the current one — otherwise last month's "you hit your target" retroactively becomes a miss.

2.3 The model's part, and what bounds it

`POST /v1/me/goals/suggest` is the second place in the system where a model produces numbers, and unlike a meal estimate these are what somebody eats to for months. Three things carry that, and none of them is the model:

1. **The formula runs first.** There is a complete, checkable answer before the model is asked anything, and it is what the screen falls back to when the call fails or is slow.
2. **The model adjusts rather than authors.** It is handed the formula's result and asked whether it should move. A model asked "*what should this person eat*" answers confidently from nothing; one asked "*here is 2,287 — should it change*" has to argue.
3. **`clampTargets` holds it.** Calories never below resting burn, protein bounded per kilo, fibre bounded outright. Every correction is returned and shown, because a figure the server moved is not the figure the model chose.

Carbohydrate and fat are never asked of the model. They follow from the calorie target, and the client derives them from whatever number is on screen so the four always add up.

The suggestion is prefetched on the step before, while the button spins, so the targets screen opens complete rather than filling in under the reader. The prefetch gives up after a few seconds; a slow model costs a spinner in one place, not a dead end.

3 Logging Module — the voice and text route

The primary route, and the one the product is judged on. Voice and keyboard are two doors into one room: both ask the same question, both end at the same read-back, and neither of them parses a word on the device.

3.1 Entry points

Entry	Surface	Why
Centre button on the tab bar	Ask sheet over the live tab	It is not a tab. Today keeps its scroll position behind it; a half-written meal is never left in the background
End of onboarding	Listen — a full screen, one orb	At that moment speaking to the app <i>is</i> the task and there is nothing else to look at
Ask sheet → keyboard	Type	The same field, filled a different way. Swapped for Listen rather than stacked on it

3.2 Speaking

Element	Behaviour
Recorder	The app's own native module (<code>com.nutricheck.recorder</code> , ~180 lines over <code>MediaRecorder</code>). No third-party native dependency
Audio	<code>VOICE_RECOGNITION</code> source — not <code>MIC</code> , which applies call-tuned AGC and noise suppression that chew the consonants a transcriber needs. 16 kHz mono, 32 kbps AAC
Retention	The clip is deleted before <code>stop()</code> resolves. A recording of somebody saying what they ate is health-adjacent and has no reason to outlive the request that consumed it
End of turn	Adaptive amplitude detection against a measured noise floor, <code>SILENCE_MS = 1800</code> — listing a meal is full of pauses. A Done button always exists ; the detector is the shortcut, not the only exit
Transcription	<code>POST /v1/transcribe</code> (Gemini). Returns text, which is shown and remains editable. A bad transcript is fixed by typing, never by re-recording
Progress	A constant sweep, never a percentage. The server answers when it answers, and this app does not draw a number it has not measured

3.3 Interpreting

`POST /v1/ai-meal` — one model call, no corpus search, per-100 g rates multiplied by our own arithmetic.

Rule	Implementation
Every number is an estimate, and says so	A summary sentence and a one-time banner above the rows; ~ on every <code>imputed</code> value. The draft type carries <code>estimated: true</code> as a literal, so a screen cannot render one without having been handed that fact
Nothing is counted on the device	No item counter, no chips split out of the sentence. English punctuation cannot count Tamil items — the old splitter read "... <i>sapten. So, how much...</i> " as two items from the comma in "So,"
The rows arrive together	One POST has no half-answer. The sheet opens immediately and echoes the phrase back; skeletons that filled in would be a progress bar for a process with no progress
Rows are marked and owned	Written <code>source: 'ai'</code> , <code>created_by_user_id</code> set, every nutrient state <code>imputed</code> — never <code>known</code> , which is the word this schema reserves for a laboratory value
Reuse, not accumulation	Keyed on <code>(source, source_id)</code> with the user inside the key, so saying "dosai" every morning reuses one row
"AI unavailable" is its own message	A missing key or a provider outage is not the same failure as an unreadable sentence, and is not reported as one

Why this route exists. The corpus holds 13,440 foods and 25 Tamil aliases. `pavakkai` finds nothing however good the trigram scoring is, because USDA files bitter gourd under *Balsam-pear*. Search-first dead-ends on the words this app's users actually say, and a dead end is worse for them than an estimate they can see is an estimate.

3.4 The resolver route (built, secondary)

`POST /v1/resolve` remains in the product and in the client (Composer → Confirm sheet). It is the corpus-grounded path: portion prefill → parse → one batched candidate search → constrained re-rank → arithmetic → SSE draft.

Step	Who does it
Parse the sentence into items and quantities	Model. No nutrient field exists in the schema
Retrieve candidates	Postgres — trigram search, top 8 per item
Pick one	Model, constrained to a per-request Zod enum of the ids Postgres just returned. An invented food is <i>unrepresentable</i> , not merely discouraged
Compute nutrition	Arithmetic — $\text{per}_{100\text{g}} \times \text{grams} \div 100$. Same input, same answer

Both routes are kept rather than folded together, so *"did this number come from a measurement"* depends on which endpoint was called, not on a branch inside one.

3.5 The confirm sheet

State	What the user sees
Confident	Food name, portion chip, macro line, quiet styling

State	What the user sees
Quantity given	The portion stated plainly — "2 rotis", "180 g" — with no range. Showing uncertainty on a number the user supplied is noise
Personal unit	"a bowl" resolved from <code>user_portions</code> , or a range on first use
No amount given	An empty, focused, dashed-amber portion chip. Asking costs one tap; guessing costs trust
Low confidence	Row flagged and expanded, runner-up candidates one tap away
Unresolved	The words that did not match, with a scoped search field
Nutrient unknown	—, not 0 g, plus exclusion from that nutrient's denominator and a per-nutrient unmeasured count
Committed	Sheet dismisses, ring animates, delta shown

Three rules hold across the whole module: **never auto-commit a parse**; **never invent an amount**; **never lose the user's words** — every failure path keeps the phrase and lands somewhere it can still be used.

3.6 Failure paths

When	What happens	Where the user lands
Offline at send	Phrase queued locally with the entry	Log appears as pending
Model times out	One silent retry, then stop	Search, phrase pre-filled
Nothing interpreted	Plain message; miss recorded with the exact words	Search, phrase kept
AI unavailable — no key or provider down	A distinct message from an unreadable sentence	Search still works
Quota exhausted	Stated plainly, with when it resets	Search and repeat still work — the app never fully stops
No corpus match	<code>match_misses</code> records the exact words	Custom-food creation, two fields
Commit fails	Entry stays in the local queue and retries	Nothing to redo

4 Search & Corpus Module

The route with no model in it. It is the floor under every failure path above, so it has to be genuinely good rather than a grudging fallback.

4.1 Search

- **Trigram over** `search_text`, GIN-indexed, blending `word_similarity` with whole-string `similarity()`. Ranking that works at 13 rows breaks at 13,440: `word_similarity` returns 1,000 for every row containing the query, so "mango" tied *Mangos, raw* with *Babyfood, fruit dessert, mango with tapioca*.
- **MIN_SCORE is a deliberate floor (0.95)**. Below it, search returns nothing rather than something wrong — "maggi" once answered with SMUCKERS MAGIC SHELL at 609 kcal. A miss is recoverable; a wrong frozen number found a week later is not.
- **Results show the numbers**, so choosing between four similar rows does not require opening each.
- **Vector search is built but dormant**. `food_embeddings` exists with an HNSW index and is empty; search is trigram-only today.

4.2 The corpus

Source	Rows	Note
USDA SR Legacy	7,793	Whole, single-ingredient foods. Carbs and fat on 100%, fibre on 92.8%
USDA FNDDS	5,431	Prepared and mixed dishes — what people name out loud
USDA Foundation	135	
Curated Indian dishes	81	The gap-filler; USDA has essentially nothing Indian, and no ranking change finds <i>dosai</i>
Total	13,440	36,768 portions, 540 aliases, 8 locales, 26 MB

Curated carbohydrate is derived, never authored. Each dish supplies one fat estimate; carbohydrate is `kcal - 4×protein - 9×fat`, clamped at zero — exactly how USDA defines *carbohydrate, by difference*. A dish with no fat estimate gets `unknown` for both rather than a guess.

4.3 The three-state rule

Carbohydrate, fat and fibre each carry their own state — **known**, **imputed** (rendered with `~`), **unknown** (rendered `-` and excluded from that nutrient's denominator) — and their own unmeasured count. The item missing fibre is usually not the item missing carbohydrate, and one shared counter could not say which total to distrust. A `CHECK` constraint in the database enforces that the state is `unknown` exactly when the value is null.

`kcal` and `proteinG` are never null. Both are reported for every corpus row and both are goal-bearing, so a missing one is a corpus bug rather than a state to render.

4.4 Custom foods and the miss log

Eight fields, blank meaning unknown, validated by the same Zod schema that shapes the request. Every failed lookup writes `match_misses` with the exact words used — searchable, groupable, and the queue that decides which dishes get curated next.

5 Today Module

The screen the app opens on, and the answer to the question people open a tracker to ask.

Element	Behaviour
Ring	Gradient calorie ring that counts down — "637 left" — and does not wrap on overshoot
Meters	Protein, carbohydrate, fat, fibre. Each carries its own "N items unmeasured" note
Meal cards	The day's entries grouped by meal slot, each with a one-line insight from <code>GET /v1/insights/meal</code> — facts from Postgres, prose from the model, no numeric field for it to invent one in
Recents	Frequency × recency × time-of-day from <code>GET /v1/suggestions/recents</code> . One tap logs at the remembered portion
Undo	A five-second toast, not a confirm dialog. Cheaper for the taps that were right, fully recoverable for the rest
Masthead	Date, opening the calendar; avatar top-right, opening the account screen
Offline queue	Commits queue in app state and drain through <code>POST /v1/logs/batch</code> , per-element results

A meal card that loses its note still has every number on it — the insight call swallows its own failures on purpose. The Ideas screen does the opposite, because there the response *is* the screen.

6 Ideas Module

A tab, between Today and Insights, because it is a place you return to: what it shows changes every time something is logged.

`GET /v1/ideas?date=&tz=` returns foods that would fit what is left of the day. It is the third route where a model produces nutrition, and the one with the weakest justification — it fires because a tab was opened rather than because somebody asked a question. Four things bound it, in this order inside `IdeasService`:

1. **The gap is computed server-side** from the same day view the Today screen renders.
2. **Every returned item is Atwater-checked** against its own macros and dropped if it fails.
3. **The model returns per-100 g rates**; the multiplication stays in our code.
4. **Every row it creates is written** `source: 'ai'`, owned by the user, all nutrient states `imputed`.

It is also the only AI route deliberately **without** `QuotaGuard`. The guard runs before the handler and therefore before the cache, so an exhausted user would be refused a list they had already been shown and already paid for. The service checks the same quota itself, after the cache lookup and before the call.

The subject is the person, not the day. The prompt puts the profile and the goal first and today's figures last, and says outright that the day is a constraint on the answer rather than the answer. Leading with the remaining targets produced a gap-filling calculator, which is not a question anyone opens an app to ask — and on a day with nothing logged it had no subject at all.

The response is cached in Redis on the gap, so the same gap twice is the same list without a second model call.

7 Insights & History Module

Screen	Shows
Insights	GET <code>/v1/logs/week?date=&tz=</code> — seven days, averages over logged days only , an uncapped streak, and a per-nutrient chart
Calendar	GET <code>/v1/logs/month?date=&tz=</code> — every day of the calendar month, logged or not, with adherence per day
Entry detail	One entry: items, portions, per-item edit, delete

Per-item portion edit (PATCH `/v1/logs/:id/items/:itemId`) refreezes the nutrients for that item **and learns the personal unit** into `user_portions`. The wholesale entry patch discards that signal, which is why the per-item route exists.

8 Account & Settings Module

Screen	Contents
You	Name and email, targets summary, profile and goal editors, change password, legal links, sign out
Profile editor	Every answer onboarding collected, editable; re-saving writes a new goal row
Goal editor	The five targets, editable directly

Reached from Today's top-right avatar rather than a tab: settings are a place you visit, not a place you live, and a permanent tab would spend the scarcest real estate on the screen to say otherwise.

9 Design System

The app is **dark, and only dark** — one cool near-black palette, no light-mode tokens and no scheme switch. Android and iOS are pinned dark natively, or the app opens on a white flash.

Token group	Values
Surfaces	<code>#0B0C0E</code> page · <code>#191D21</code> cards · blue-violet accent
Cards	A fill and a radius. No hairline, no shadow: a card's edge is the step in lightness between its fill and the page, which on a dark screen is the only depth cue that carries

Token group	Values
Buttons	No button is the accent colour — ink on canvas, everywhere, one treatment. Selection is ink too
Accent	Reserved for work: a focused field, a filled ring, the mark
Radius	xs 8 · sm 12 · md 16 · lg 20 · xl 28 · pill
Type	One platform sans, nine roles, reached only through <code><Text role="..."></code> . Brand face is a one-line swap
Motion	Springs for finger-initiated, durations for system-initiated

The rule that outranks aesthetics: amber never decorates. If something is amber, the app is saying it does not know something — unmeasured fibre, a portion nobody stated, a low-confidence match. Spending amber on a highlight would make the one signal that protects the product's credibility unreadable.

Every pressable routes through one `Press` component (spring to 97.5%, optional semantic haptic). Every food row carries a `FoodGlyph` whose tint is hashed from the food id, so it is stable across screens and sessions — it is what stops the lists reading as a spreadsheet.

Forms are `react-hook-form` + `Zod`. Screens hold no field state and write no validation; bounds are *copied* from the server contract with the twin named in a comment, because a rule only the client enforces is a field the user cannot fill, and a rule only the server enforces is a 422 they cannot read.

10 AI Layer

Six model-backed capabilities, one pluggable provider, and one rule with three documented exceptions.

10.1 The rule

The model never emits a nutrient value — except on the three routes below, where it does so as a rate, bounded, marked `imputed`, and multiplied by our own arithmetic.

Capability	Route	Model produces	Guardrail
Parse	<code>POST /v1/resolve</code>	Items and quantities	No nutrient field exists in the schema
Re-rank	<code>POST /v1/resolve</code>	One food id	Per-request <code>Zod</code> enum of ids Postgres just returned
Whole meal	<code>POST /v1/ai-meal</code>	Foods + per-100 g rates	\$3.3 — rates not totals, rows marked and owned, states <code>imputed</code>
Targets	<code>POST /v1/me/goals/suggest</code>	An adjustment to a computed answer	\$2.3 — formula first, <code>clampTargets</code> after
Ideas	<code>GET /v1/ideas</code>	Foods + per-100 g rates	\$6 — server-computed gap, Atwater check, cache, self-checked quota
Meal insight	<code>GET /v1/insights/meal</code>	Prose only	No numeric field in the response

Capability	Route	Model produces	Guardrail
Transcription	POST /v1/transcribe	Text	Returns text, never a draft. The only route that accepts audio

10.2 Provider and cost

`AIService` is an abstract class with two implementations — Anthropic, and any OpenAI-compatible host — which made the provider swap nearly free and lets the whole pipeline be tested without a network.

Measure	Value
Running configuration	<code>openai-compatible</code> on <code>gpt-4o-mini</code>
Resolve	\$0.000385 , ~2.2 s
Whole meal (<code>/v1/ai-meal</code>)	\$0.000256 — 869 input, 209 output tokens
Accounting	Every call writes <code>ai_runs</code> : step, prompt version, model, tokens, cache reads, cost, latency, stop reason
Quota	<code>RESOLVE_DAILY_QUOTA=50</code> per user per day
Spend ceiling	<code>RESOLVE_USER_DAILY_SPEND_USD=1.00</code> per user per day, computed from actual token usage

The API key never ships in the app. Every model call proxies through the backend, which is where quota, abuse limits, cost attribution and prompt versioning live. A maximum phrase length guards against a 4,000-word "meal" probing for a free model.

10.3 Caching

The phrase cache key carries the prompt version (a content hash, so it cannot be forgotten), the model, **and** `RESOLVER_VERSION` — a code change to resolution logic is otherwise invisible for the 24-hour TTL. Prompt caching has a provider-specific floor: Claude will not cache a prefix under 512 tokens and OpenAI's automatic caching needs ≥ 1024 , silently in both cases. `packages/prompts` exports `approximateTokens` for exactly this check.

11 Background Behaviour & Automations

Automation	Trigger	Action
Offline queue	A commit with no connection	The entry, its <code>clientId</code> and its phrase are queued and drained through <code>POST /v1/logs/batch</code> on reconnect
Idempotent commit	A replayed queue element	<code>UNIQUE (user_id, client_id)</code> — the second write returns the first result rather than duplicating the meal
Circuit breaker	Repeated provider failure	The resolver opens the breaker and degrades to search rather than hanging

Automation	Trigger	Action
Quota refill	Daily	Redis token bucket; GET /v1/quota reports remaining resolves and the reset time
Spend ceiling	Per-user daily ai_runs.cost_usd cap	Alerts before it blocks
Refresh rotation	Every refresh	Rotating opaque token, SHA-256 stored; reuse detection revokes the whole family
Response prune	Nightly	ai_runs.response nulled at 30 days; the row's metrics persist. Rows deleted at 13 months
Draft expiry	1 hour	Redis TTL. A draft is not a log
Phrase cache	24 hours	Redis TTL, keyed as \$10.3
Deploy	Push to staging	GitHub Actions builds one image, runs migrations as a pre-deploy step, restarts behind Caddy

12 Database Schema (PostgreSQL)

One database, shared by the API and the ingest tooling. Migrations 0000 – 0009 applied.

Group	Tables and key columns
Corpus	foods (id, source, source_id, name, brand, is_generic, search_text) · food_nutrients (kcal, protein_g — never null; carbs_g / fat_g / fiber_g each nullable with its own *_state) · food_portions (label, grams, is_default) · food_embeddings (vector(384), HNSW) · food_aliases
Identity	users · user_profiles (name, sex, age, height, weight, activity_level, objective) · auth_identities · refresh_tokens
Targets	goals (kcal, protein_g, carbs_g, fat_g, fiber_g, effective_from, basis jsonb) — append-only
Personalisation	user_portions (unit_label, food_id, grams, n_corrections) · user_phrases · user_foods · recipes · meals / meal_items
Logs	log_entries (client_id, logged_at, meal, source text voice search repeat photo, phrase, ai_run_id) · log_items (food_id, grams, and a frozen copy of every nutrient with its state, quantity_type, quantity_source)
Operations	ai_runs (prompt_version, model, step, input_hash, cached, tokens, cost_usd, latency_ms, response jsonb) · match_misses · ai_food_matches

Design decisions

- **Nutrients are frozen at commit.** log_items stores the computed values rather than recomputing from foods on read. It looks like denormalisation for speed; it is really about truth — USDA reissues data, and computed history would silently rewrite a Tuesday in March. A test mutates the corpus and asserts the entry is unchanged.

- **Every nutrient state is NOT NULL , and the pairing is a CHECK constraint.** Nullable invites a coalesce in a serialiser, and a day's denominator is then quietly wrong. Every write site must state which of the three cases applies.
- **goals is append-only.** A day view resolves the goal in effect on that date.
- **log_source includes 'photo' from the first migration.** Nothing writes it. An unused enum value costs nothing; ALTER TYPE on a hot enum in production is not free — as migration 0007 proved when it rebuilt activity_level and lost the rows holding the dropped value.
- **ai_food_matches is a quarantine, deliberately not food_aliases.** That table is human-authored and is what search scores against; mixing model output into it makes *"who wrote this"* unanswerable a month later. A null food_id there is the valuable state, not the failure one: the model understood the word and we genuinely do not stock the food — the dish backlog arriving as data.
- **ai_runs.user_id is ON DELETE SET NULL.** Spend history survives a deleted account; the meal phrase does not.

13 User Roles & Permissions

There is **one role**: the signed-in user. NutriCheck is a single-tenant consumer app with no staff console, no admin panel and no shared data — which is a scope statement, not an omission.

Concern	Implementation
Authentication	Email + password only. Argon2id (m=19456, t=2, p=1) per the OWASP cheat sheet
Password policy	Length only — minimum 6, maximum 200. Composition rules push people to Password1! ; the maximum is a hashing-DoS guard
Account enumeration	One error for unknown-account and wrong-password, with a dummy Argon2 verify on the unknown path so the timing matches
Access token	JWT, 15 minutes, HS256
Refresh token	Opaque, 30 days, rotating, stored as a SHA-256 hash; reuse revokes the family
Authorization	JwtAuthGuard global, @Public() opts out. Every query is scoped by userId at the repository layer
Rate limiting	Per-IP on auth, per-user elsewhere. All windows ten minutes; registration 30 per window — Indian mobile carriers put thousands of subscribers behind one address, so a stricter per-IP limit refuses genuine people
Change password	Revokes every session on every device
Account deletion	Cascades to logs, drafts, user_portions and phrases; exercised by an end-to-end test
PII	Meal phrases are treated as health-adjacent: debug -level logging only, excluded from error reports, deleted with the account. Audio never persists
Third-party auth	Apple and Google are in the auth_provider enum and not in this build . Apple becomes mandatory on iOS the moment any social login exists

Permissions on device. The microphone is asked for at the first press of the mic button, framed as "to log by speaking", and typing works if it is denied. Nothing else is requested: no notifications, no health, no camera. A user can install, onboard, set targets and log every meal **without granting a single system permission** — an unusual position for a health app and worth protecting.

14 Integration Specifications

Integration	Use	Notes
OpenAI-compatible provider (<code>gpt-4o-mini</code>)	Parse, re-rank, whole-meal read, targets suggestion, ideas, meal insight	What actually runs. Structured outputs; <code>AI_STRICT_SCHEMA=true</code> is what makes an invented food unrepresentable — it is opt-out for a reason
Anthropic (<code>claude-opus-5</code>)	The same six steps	Built and pluggable; <code>.env.example</code> and <code>docs/PLAN.md</code> still record it as the provider while <code>openai-compatible</code> is what runs. That contradiction is unresolved
Google Gemini	<code>POST /v1/transcribe</code>	A separate provider abstraction from the resolver's, on purpose: different vendor, different billing unit, different failure modes — and the resolver keeps working when this is not configured
USDA FoodData Central	Corpus ingest — SR Legacy, FNDDS, Foundation	Pinned and checksummed download. FDC references nutrients by surrogate id in SR Legacy and by legacy <code>nutrient_nbr</code> in FNDDS; the mismatch silently ingested zero of 5,432 FNDDS rows until it was caught
Open Food Facts	Packaged goods	Not ingested. ODbL share-alike review never opened — a launch blocker if found late
PostgreSQL 16 + <code>pgvector</code> + <code>pg_trgm</code>	Corpus, logs, identity	
Redis	Rate limits, quota bucket, draft store, phrase cache, ideas cache	Throttler storage is still in-memory rather than Redis — per-pod counters mean the login limit multiplies by the replica count
AWS Lightsail + Caddy	Staging	4 GB instance, HTTPS via a real Let's Encrypt certificate on an <code>sslip.io</code> hostname that encodes the static IP
GitHub Actions	Build, migrate, deploy on push to <code>staging</code>	Runs no tests — a deliberate call recorded in <code>docs/CI-CD.md</code>

15 Build Order & Status

"Built (local)" means complete and verified on a developer machine but not deployed to staging.

Stage	Status
Foundations — workspace, Docker, migrations, config validation, RFC 9457 errors, health probes	Built · deployed
Auth — register, login, rotating refresh with reuse detection, change password	Built · deployed
Corpus & ingest — 13,440 foods, 36,768 portions, 540 aliases, trigram search	Built · deployed
Goals — Mifflin–St Jeor, append-only, five targets	Built · deployed
Logs — commit with frozen nutrients, idempotency, batch drain, day view, per-item edit that learns the unit	Built · deployed
Week summary, month view, recents, saved meals and phrases	Built · deployed
Resolver (M2) — prefill → parse → batched search → constrained re-rank → arithmetic → SSE draft	Built · deployed
Transcription — <code>POST /v1/transcribe</code> , Gemini	Built · deployed
Corpus-free path — <code>POST /v1/ai-meal</code>	Built · deployed
Meal insight, ideas	Built · deployed
Mobile — onboarding, Today, search, confirm, entry, insights, calendar, account	Built (local)
Mobile — the voice route (ask sheet, listen, type, meal details)	Built (local) — verified on a physical device end to end, including a Tamil phrase
Dark-only redesign, merged auth flow, six activity levels, model-suggested targets	Built (local) — 58 commits unpushed (see Open Item #1)
Weight tracking and trend (M3)	Not started
Password reset	Not started — a forgotten password is currently an unrecoverable account
Eval harness	Not started — the highest-value missing thing
Tests in CI	Not wired, deliberately
Offline persistence	Not built — the queue is in-memory, so a cold start loses queued commits
Embeddings + hybrid ranking	Table and index exist, empty
<code>identify()</code> + <code>ai_food_matches</code>	Built and unreachable — no route
Photo logging	Parked by design, not missing. The <code>photo</code> enum value and the one-adapter shape are held open for it
Store submission (M4)	Not started

Test counts (as recorded 2026-08-28): API 89 unit + 133 integration, green. Mobile 107 across seven suites — **not run since the theme rewrite, the auth merge and the onboarding rebuild landed**, by instruction.

Route count (counted 2026-08-28): 40 under `/v1`, plus 3 health probes.

16 Acceptance Criteria

- **Onboarding:** five steps, one question each; the targets screen opens with numbers already in hand; every target is editable; the calorie target never falls below BMR and the screen says so when it is floored.
 - **Targets:** the formula's answer is complete before the model is asked; a model figure outside `clampTargets` is corrected and the correction is shown; carbohydrate and fat always add up to the calorie target on screen.
 - **Voice:** the microphone never starts because a screen opened — only because a person pressed it; a Done button always ends the turn and keeps what was said; the transcript is shown and remains editable before anything is interpreted; the audio clip does not outlive the request.
 - **Interpretation:** no item count is derived on the device; every number on an `ai-meal` draft renders as an estimate; a missing key or a provider outage produces a different message from an unreadable sentence.
 - **Commit:** `POST /v1/resolve` and `POST /v1/ai-meal` write no log entry; only `POST /v1/logs` commits; the same `clientId` twice produces one entry; nutrients on a committed entry do not change when the corpus is re-ingested.
 - **The three-state rule:** an unknown nutrient renders `-`, never `0 g`; it is excluded from that nutrient's denominator; each of carbohydrate, fat and fibre carries its own unmeasured count.
 - **Never invent an amount:** a phrase with no quantity produces `none_given` and an empty, focused portion chip; an unlearned personal unit produces a range, never a number.
 - **Search:** a query below the score floor returns nothing rather than a plausible wrong food; every miss is recorded with the exact words; the miss lands on custom-food creation, not a dead end.
 - **Portion edit:** editing one item's portion refreezes that item and writes the personal unit to `user_portions`.
 - **Day and history:** the ring counts down and does not wrap on overshoot; a day view resolves the goal that was in effect on that date; week averages count logged days only.
 - **Ideas:** every returned item passes an Atwater check against its own macros; an exhausted user still sees a list they have already paid for; a failed call is named on screen rather than swallowed.
 - **Security:** every query is scoped by `userId` at the repository layer; a refresh-token reuse revokes the family; no API key reaches the device; meal phrases never appear above `debug` in logs.
 - **Failure:** every documented failure path keeps the user's words and lands on a route that cannot fail.
-

17 Open Items & Decisions

#	Item	Owner	Priority
1	58 unpushed commits, and migration 0007 loses data. It rebuilds <code>activity_level</code> to drop 'active', folding those rows into 'moderate'; 0008 puts the value back but not the rows. Squash 0007 / 0008 into a single <code>ADD VALUE 'athlete'</code> , or accept the loss on staging after checking the row count	Dev	High
2	The eval harness. Without it, "the model picked the wrong chicken" is an anecdote rather than a number, and every prompt edit and model change ships ungated	Dev	High
3	Run the mobile suite. It has not run across the theme rewrite, the auth merge and the onboarding rebuild — <code>screens.test.tsx</code> is the natural net for exactly those	Dev	High
4	Password reset does not exist. Email + password with no recovery means a forgotten password is a lost account. Building it means choosing an email transport	Dev / Product	High
5	Offline persistence. The commit queue is in-memory, so a cold start loses queued entries — which contradicts "nothing is ever lost"	Dev	High
6	/v1/ai-meal prompt quality. The summary restates the sentence instead of giving the energy figure the prompt asks for; coconut chutney came back at 100 kcal/100 g against a real ~190	Dev	Medium
7	The targets prompt. Caught once returning 2,280 against a calculated 2,294 and calling it "adjusted slightly" — fourteen calories offered as advice. Forbidden by the prompt and snapped by the server now; this is the failure mode to watch	Dev	Medium
8	~100 Tamil aliases. 25 of 7,928 USDA rows carry one, and that number is the entire reason <code>/v1/ai-meal</code> exists. Hand-written aliases fix the common cases with no model and no invented numbers	Ops / Dev	Medium
9	Deepen Tamil, or build recipe decomposition? Only 34 of 81 curated dishes are Tamil-reachable. Deepening is direct but every dish is a research task; decomposition derives dishes from real USDA ingredient rows	Product	Medium
10	Curated kcal and protein carry no honesty marker. Fibre signals estimation via <code>imputed</code> ; a curated dish's calories look exactly as authoritative as a measured USDA row. The biggest honesty gap in the corpus	Product / Dev	Medium
11	Legal links are unverified. <code>src/lib/legal.ts</code> points at <code>nutrheck.app/privacy</code> and <code>/terms</code> ; nothing is published there. "You accept these" beside a link that 404s is worse than either half alone	Product	Medium
12	Open Food Facts share-alike review never opened. A launch blocker if found late	Legal / Product	Medium
13	Free, subscription or freemium? The paid boundary is not obvious — unlimited AI interpretation versus a monthly cap, history depth, or insights. It shapes onboarding and gating, so it is decided before it is built	Management	Medium
14	Free-tier quota and spend ceiling are 50/day and \$1/day — both guesses, unvalidated against real usage	Product	Medium
15	Barcode scanning decides whether the app asks for camera permission at all, which is currently its cleanest privacy claim	Product	Low

#	Item	Owner	Priority
16	Social login. Email + password only today; Apple becomes mandatory on iOS the moment Google is added	Product	Low
17	Insight cost accounting. <code>/v1/insights/meal</code> calls the model and records nothing, so its spend is invisible to the ceiling. The <code>ai_step</code> enum already has the value	Dev	Low
18	eslint has never run — it is in no <code>devDependencies</code> . Prettier is, but the repo is not Prettier-formatted, so running it rewrites whole files	Dev	Low
19	Throttler storage is in-memory , not Redis, so per-pod counters multiply the login limit by the replica count	Dev	Low
20	Documentation drift: <code>.env.example</code> and <code>docs/PLAN.md</code> §3 record anthropic as the provider while <code>openai-compatible</code> runs; <code>docs/BACKEND.md</code> §12 records a 10-character password minimum against a live minimum of 6	Dev	Low

Decisions already taken — recorded so they are not relitigated without reason. Photo logging is parked, not cancelled, and the architecture holds one adapter-shaped hole for it. Language scope is Tamil and English; Hindi, Bengali and Gujarati rows stay because they cost nothing and much of that food is everyday eating in Tamil Nadu. Logging is a raised centre action, not a tab. The app is dark only. The mock backend is deleted — there is one implementation of the API client.

18 Approvals & Sign-off

Reviewed and approved by:

Project Owner

Signature: _____ Date: _____

Project Developer (AI Team)

Signature: _____ Date: _____

Project Manager (AI Team)

Signature: _____ Date: _____

Product Manager

Signature: _____ Date: _____

Nutrition Lead

Signature: _____ Date: _____

General Manager

Signature: _____ Date: _____

*NutriCheck provides informational estimates and is not medical advice; the app is not a medical device.
Nutrition figures marked with ~ are estimates rather than measured values.*